

Panduan Lengkap: AI Prompting untuk Developer (Context Engineering)

Versi: 2026 Edition
Berdasarkan: Andrej Karpathy, Anthropic, OpenAI, Hugging Face, Braintrust
Target: Software Engineer, Mentor, Student yang ingin menguasai AI sebagai pair programmer

Filosofi Utama: Context Engineering, Bukan Prompt Engineering

"LLM adalah CPU. Context window adalah RAM. Tugas kamu adalah operating system -- memuat working memory dengan data dan instruksi yang tepat untuk setiap task." -- Andrej Karpathy, Juni 2025 [^2^]

Prompt Engineering sudah mati. Yang menggantikannya adalah Context Engineering -- seni merakit konteks yang tepat, bukan sekadar menulis kalimat cantik.

Perbedaan mendasar:

Prompt Engineering (Lama)	Context Engineering (2026)
Fokus pada "magic phrase"	Fokus pada konteks, constraint, dan evaluasi
One-shot, berharap sempurna	Iteratif, tracked, versioned
Tanya -> Dapat jawaban	Design interface -> Test -> Refine
Casual user mindset	Production system mindset

Filosofi untuk Developer:

"Treat prompting as system design. You're defining interfaces, constraints, failure modes, and evaluation criteria." [^1^]

Framework Prompting: Pilih Sesuai Kebutuhan

A. RTF Framework -- Untuk Task Cepat (Debug, Review, Dokumentasi)

Role	-> "You are a senior Laravel backend engineer."
Task	-> "Review this API endpoint for security issues."
Format	-> "List findings in a table: Issue Severity Fix."

Kapan pakai: Debugging, summarizing logs, generate docs, code review cepat. [^3^]

B. RASC Framework -- Untuk Analisis & Strategi

Role	-> "You are a product security architect."
Action	-> "Analyze authentication flow and suggest improvements."
Steps	-> "1. Identify vulnerabilities. 2. Rank by risk. 3. Propose fixes."

Context -> "We use Laravel Sanctum, MySQL, deployed on AWS EC2."

Kapan pakai: Arsitektur decision, security audit, feature planning. [^3^]

C. COSTAR Framework -- Untuk Output yang Kompleks & Terstruktur

Context -> "The app is a blog API with 1000+ daily active users."
 Outcome -> "Reduce API response time by 30%."
 Style -> "Step-by-step breakdown with code examples."
 Tone -> "Professional, technical, prioritize clarity."
 Audience -> "Backend developers familiar with Laravel."
 Response -> "Markdown list with priority levels and estimated impact."

Kapan pakai: Technical specification, refactoring plan, performance optimization. [^3^]

Struktur Prompt yang Efektif (Universal Template)

Setiap prompt produksi harus memiliki 5 komponen ini:

1. Role Definition (Siapa AI-nya?)

Buruk: "Buatkan saya kode API."
 Bagus: "You are a senior Laravel engineer specializing in REST API design and security. You prioritize clean code, proper validation, and authorization checks."

4 bagian role yang efektif: [^1^]

- Function -- "Review this endpoint for production readiness."
- Perspective -- "You are a Laravel Sanctum expert."
- Audience -- "Assume I have 3 months of PHP experience."
- Scope -- "Focus on validation and error handling, NOT code style."

2. Task & Context (Apa yang harus dikerjakan?)

Buruk: "Buatkan migration."
 Bagus: "I am building a Smart Blog API using Laravel 11, PHP 8.3, MySQL 8.0, and Sanctum for authentication.

 Current state: Users table already exists.
 Next: I need to create posts, comments, tags, and post_tags tables.

 Requirements:
 - Proper foreign keys with cascade delete
 - Timestamps automatic
 - Index for performance"

Prinsip: Generic prompts produce generic answers. Stack details change everything. [^1^]

3. Structured Output Format (Format output seperti apa?)

Buruk: "Jelaskan error ini."

Bagus: "Return ONLY a JSON object with these exact fields:

```
{
  'error_type': '...',
  'root_cause': '...',
  'fix_code': '...',
  'prevention': '...'
}
```

Do not include any explanation outside the JSON."

Format yang bisa diminta: `[^1^][^5^]`

- JSON -> untuk integrasi dengan sistem/workflow
- Markdown table -> untuk perbandingan/analisis
- Numbered checklist -> untuk step-by-step
- Code block with comments -> untuk kode

4. Constraints & Boundaries (Batasan apa?)

"Constraints:

- Do NOT use mock data. Only real implementation.
- Do NOT include deployment steps. Focus on code only.
- Maximum 50 lines per function.
- Use Laravel best practices (Eloquent, not Query Builder)."

Jenis constraint yang efektif: `[^1^]`

- Scope -- "Database design only, not deployment."
- Audience -- "Explain for a beginner who knows OOP but not Laravel."
- Format -- "Return numbered plan, one example per step."
- Decision -- "Recommend ONE option and justify why others are weaker."

5. Few-Shot Examples (Contoh "good" output)

"Here are examples of the code style I want:

Example 1 (Good):

```
public function store(Request $request)
{
    $validated = $request->validate([
        'title' => 'required|string|max:255',
    ]);
    return response()->json(['data' => $validated], 201);
}
```

Example 2 (Bad - what I DON'T want):

```
public function store(Request $request) {
    $post = new Post();
    $post->title = $request->title; // No validation!
    $post->save();
}
```

```
return $post;  
}
```

Now, generate a CommentController following the GOOD pattern above."

Fakta menarik: Bahkan contoh dengan label yang salah (randomized) masih outperform zero-shot. Yang penting adalah diversity input space, bukan kesempurnaan contoh. [^2^][^6^]

Teknik Advanced

1. Chain-of-Thought (CoT) -- Untuk Task Kompleks

Gunakan saat: Schema design, debugging, architectural trade-offs, algorithm.

Jangan gunakan saat: Simple extraction, formatting, atau reasoning models (o-series, Claude Extended Thinking, Gemini Thinking Mode) -- mereka sudah reasoning internal. [^2^][^6^]

```
Buruk: "Design the best database schema for e-commerce."  
Bagus: "Design a database schema for e-commerce. Follow this reasoning:  
  
    Step 1: Identify core entities and relationships.  
    Step 2: Define constraints and normalization level.  
    Step 3: Analyze query patterns and add indexes.  
    Step 4: Recommend final schema with trade-offs."
```

Hasil: Structured outputs separating reasoning from decisions dapat mengurangi error rate hingga 25% pada task kompleks. [^1^]

2. Meta Prompting -- Untuk Pattern yang Berulang

Gunakan saat: Task serupa yang berulang, ingin hemat token. [^6^]

```
"For any coding task I give you, follow this structure:  
  
Step 1: Identify the problem and constraints.  
Step 2: Write the function with input validation.  
Step 3: Add error handling with proper HTTP status codes.  
Step 4: Provide one usage example.  
  
Now apply this to: [task spesifik]"
```

3. Iterative Refinement -- Workflow Produksi

Prompt pertama tidak perlu sempurna. Workflow yang benar: [^1^][^2^]

```
Iterasi 1 (Broad):    "Explain REST API design."  
Iterasi 2 (Focused): "Explain REST API design for beginners using Laravel examples."  
Iterasi 3 (Constrained): "Focus on CRUD endpoints, validation, and common mistakes.  
                        Return a numbered plan with one code example per step."  
Iterasi 4 (Polished): "Add authorization checks to each endpoint example."
```

Rule: Save your best iteration chains. Treat them like reusable assets -- internal tooling, bukan sekadar chat history. [^2^]

Model-Specific Tactics (2026 Update)

Claude (Anthropic) -- XML Tags & Literal Instructions

- Gunakan XML tags: <instructions>, <context>, <example>, <format>
- Hindari aggressive language: "CRITICAL!", "YOU MUST", "NEVER EVER" -> overtrigger dan hasil lebih buruk
- Gunakan adaptive thinking mode untuk task kompleks
- Prompt caching: Letakkan konten statis di awal (system instructions, few-shot) dan variabel di akhir -- hemat cost hingga 90% [^2^]

```
<role>
You are a senior Laravel security engineer.
</role>

<context>
I am building a blog API using Laravel 11 and Sanctum.
</context>

<task>
Review this AuthController for security vulnerabilities.
</task>

<format>
Return a Markdown table: Vulnerability | Risk Level | Fix Code
</format>
```

GPT-5 (OpenAI) -- Conversational, Skip Explicit CoT

- GPT-5 adalah router-based system -- multiple model di belakang satu endpoint
- Jangan tambahkan "think step by step" -- bisa trigger wrong model
- Zero-shot first -- GPT-5 surprisingly good at inferring intent
- Pin production apps ke specific model snapshot -- router behavior berubah antar versi [^2^]

```
"Review this Laravel API code for SQL injection risks.
Suggest fixes with secure alternatives."
-> Cukup. Tidak perlu "think step by step" atau role panjang.
```

Gemini (Google) -- Shorter & More Direct

- Prefer shorter, more direct prompts daripada Claude/GPT
- Few-shot examples WAJIB -- zero-shot explicitly NOT preferred
- Letakkan pertanyaan spesifik di AKHIR prompt, setelah data context
- Excellent untuk very long/structured responses, tapi butuh constraint jelas [^2^][^5^]

Task: Identify 3 performance bottlenecks.
Format: Numbered list with line references."
-> Pertanyaan di akhir, pendek, langsung.

Anti-Pattern: Apa yang Harus Dihindari

1. Prompt Buruk (Vague)

```
"Buatkan saya blog API"  
"Fix this code"  
"Explain Laravel"
```

Masalah: AI bingung scope, stack, audience, format. Hasil generic, sering salah. [^1^]

2. Negative Framing

```
"Don't use mock data. Don't forget validation. Don't make it complex."
```

Masalah: Pink Elephant Problem -- otak (dan AI) harus memproses konsep negatif dulu.

Solusi: Reframe ke positive: "Use real implementation. Include validation. Keep it simple." [^1^][^2^]

3. Over-Reliance pada Conversational Memory

```
"Ingat ya kemarin kita bikin..."
```

Masalah: AI lupa, kontradiksi, drift.

Solusi: Treat context like project state. Simpan project brief dan paste ulang di setiap sesi baru. [^1^][^2^]

4. No Output Structure

```
"Apa pendapatmu tentang approach ini?"
```

Masalah: Free-form prose sulit diparse, dipakai, atau diintegrasikan.

Solusi: Selalu minta format: JSON, Markdown table, numbered list, atau code block. [^5^]

Production Best Practices (Untuk Tim)

1. Version Control Your Prompts

"If your prompt runs more than once, it belongs in version control." [^2^]

Prompt drift is real -- kamu tweak sesuatu di Kamis sore, lupa apa yang diubah, dan Senin pagi debug output yang tadinya jalan.

2. Build a Golden Test Set

Simpan representative inputs dengan expected outputs. Run pada setiap perubahan prompt. Ini regression testing untuk instruksi yang menghasilkan kode. [^2^]

3. Prompt Compression

Hapus filler words, ubah ke directive format:

Verbose	Compressed
"Could you please make sure to write a friendly error message"	"Task: Friendly error message."
"We'd like the tone to be warm, approachable, and also professional"	"Tone: warm, professional."

Hemat token ~60% tanpa kehilangan presisi. [^5^]

4. Structure for Caching

Letakkan konten statis di AWAL prompt (system instructions, few-shot examples, tool definitions) dan konten variabel di AKHIR (user query, data spesifik).

Hasil: Anthropic prompt caching bisa hemat cost 90% dan latency 85%. [^2^]

5. Prompt Scaffolding for Security

Untuk production apps yang terpapar public input:

- Layer defenses: system messages + output constraints + guardrails
- Jangan biarkan user input overwrite system instructions
- Test adversarially (red teaming) [^5^]

Cheat Sheet: Template Siap Pakai

Template A: Generate Code (Laravel/Backend)

```
<role>
You are a senior [FRAMEWORK] engineer. You write clean, secure,
production-ready code with proper validation and error handling.
</role>

<context>
- Stack: [PHP/Laravel/Node/etc] version [X]
- Current state: [apa yang sudah ada]
- Database: [MySQL/PostgreSQL/etc]
- Auth: [Sanctum/JWT/etc]
</context>

<task>
[Deskripsikan task spesifik]
</task>
```

```
<constraints>
- Do NOT use mock data
- Include input validation
- Return consistent JSON response format
- Add authorization checks where needed
- Maximum [X] lines per function
</constraints>

<format>
Return ONLY the code file content. No explanation outside code comments.
</format>
```

Template B: Debug & Fix

```
<role>
You are a debugging expert specializing in [LANGUAGE/Framework].
</role>

<context>
I got this error while running [command/context]:
</context>

<error>
[Paste error message lengkap]
</error>

<code>
[Paste kode sekitar error, minimal 10 baris before & after]
</code>

<task>
1. Identify the root cause
2. Provide the fixed code
3. Explain why the error happened (1 sentence)
4. Suggest how to prevent this in the future
</task>

<format>
Return as Markdown with these sections: Root Cause | Fixed Code | Explanation | Prevention
</format>
```

Template C: Code Review

```
<role>
You are a [SECURITY/PERFORMANCE/GENERAL] code reviewer.
Focus on: [VALIDATION/AUTH/PERFORMANCE/READABILITY].
Ignore: [STYLE/NAMING/FORMATTING].
</role>

<code>
[Paste code]
</code>

<task>
```

Review this code and identify issues ranked by severity.

For each issue, provide: location, problem, fix code.

</task>

<format>

Markdown table: Issue | Severity | Location | Fix

</format>

Template D: Learning/Explanation

<role>

You are a patient mentor explaining [TOPIC] to a [BEGINNER/INTERMEDIATE] developer who knows [PREREQUISITE] but not [TOPIC].

</role>

<task>

Explain [CONCEPT] using:

1. One real-world analogy
2. One simple code example
3. One common mistake to avoid
4. One practice exercise I can try

</task>

<format>

Numbered sections. Keep each under 100 words. Use [LANGUAGE] syntax.

</format>

Contoh Prompt untuk Developer (Copy-Paste Ready)

1. Generate Migration

You are a Laravel database architect.

I need migration files for:

1. posts (id, author_id FK->users, title, content, status enum, timestamps)
2. comments (id, post_id FK->posts, author_id FK->users, content, timestamps)
3. tags (id, name unique, slug unique)
4. post_tags (id, post_id FK, tag_id FK, unique composite)

Requirements:

- Cascade delete on all foreign keys
- Index on author_id, post_id, status
- Timestamps automatic

Return ONLY the migration file contents. No explanation.

2. Generate Controller dengan Auth

You are a senior Laravel API engineer.

Create a PostController with:

- index: list published posts with pagination (10 per page)

- store: create post (title, content, status) -- require login
- show: detail post with author and comments
- update: edit post -- ONLY author can edit (return 403 otherwise)
- destroy: delete post -- ONLY author can delete
- search: search by title/content keyword

Stack: Laravel 11, Sanctum, MySQL.

Format: Consistent JSON response {success, message, data}.

Return ONLY the controller code.

3. Debug Error

You are a Laravel debugging expert.

Error:

SQLSTATE[HY000]: General error: 1824 Failed to open the referenced table 'posts'

Context: Running php artisan migrate. The comments migration has a foreign key to posts.

Question:

1. What causes this?
2. How to fix it?
3. How to prevent it in the future?

Return answer in Markdown table: Cause | Fix | Prevention

4. Review Security

You are a security-focused code reviewer.

Review this Laravel code for vulnerabilities:

[PASTE CODE]

Focus on: SQL injection, mass assignment, auth bypass, validation bypass.

Ignore: code style, naming conventions.

Return: Markdown table with Vulnerability | Severity | Fix Code

Checklist Sebelum Kirim Prompt

- ☐ Role sudah jelas? (Siapa AI-nya?)
- ☐ Stack/tech context sudah disebutkan? (Laravel 11, PHP 8.3, MySQL 8.0)
- ☐ Task spesifik? (Bukan "buatkan API" tapi "buatkan PostController dengan fitur X")
- ☐ Format output sudah ditentukan? (JSON, Markdown table, code block)
- ☐ Constraints sudah ada? (Jangan mock data, max 50 baris, fokus security)
- ☐ Few-shot example disertakan? (Kalau pattern penting)
- ☐ Negative instruction di-reframe ke positive? ("Don't" -> "Use/Only/Always")
- ☐ Scope dibatasi? ("Jangan jelaskan deployment, fokus kode saja")

Referensi

- [1] Codeling.dev -- Master Prompt Engineering Best Practices for 2026 AI (2026-04-20)
 - [2] Thomas Wiegold -- Prompt Engineering Best Practices 2026 (2026-02-21)
 - [3] Codewave -- AI Prompt Engineering Cheat Sheet for Software Teams (2026-03-18)
 - [4] KMS Technology -- 30 ChatGPT Prompts for Software Development Engineers (2026-04-14)
 - [5] Lakera.ai -- The Ultimate Guide to Prompt Engineering in 2026 (2025-06-17)
 - [6] K2View -- Prompt Engineering Techniques: Top 6 for 2026 (2024-09-11)
 - [7] Pluralsight -- Prompt Engineering 101 for Developers (2024-04-15)
-

"Stop asking, 'How do I get a better answer?' Start asking, 'How do I make good answers repeatable?'" [1]